

Reasoning

Zachary R. McCaw

August 2026

Reasoning Frameworks

Chain-of-thought prompting

Wei *et al* [30] introduce chain-of-thought (CoT) prompting, in which a language model is prompted with a small number of demonstrations that include intermediate reasoning steps rather than only the final answer. They argue that CoT reasoning encourages complex problems to be decomposed into simpler steps, can facilitate symbolic manipulation, and produces an auditable sequence of intermediate reasoning steps. CoT prompting (including the question, intermediate reasoning steps, and final answer) is compared with standard prompting (including only the question and final answer). They find that the benefits of CoT prompting emerge only after a model reaches sufficient scale.

Self-consistency

Wang *et al* [29] introduce self-consistency, a decoding strategy that replaces greedy decoding with multiple sampled reasoning paths. The model is prompted with CoT examples and then generates a set of candidate solutions using stochastic decoding. Each path produces a final answer, and the reasoning paths are “marginalized” by selecting the answer that appears most often. The method requires no additional training, fine-tuning, human annotations (beyond the initial examples), or auxiliary verifier. Note that self-consistency regards the reasoning paths as ancillary to the final answer. There is no attempt to reconcile, merge, or otherwise ensure logical compatibility among paths.

Self-taught reasoner

Zelikman *et al* introduce STaR (Self-Taught Reasoner) [38], an iterative self-training method that bootstraps improved reasoning from a small set of gold-standard examples. The model uses these examples as few-shot context to generate rationales and answers for a larger dataset, retaining outputs that reach the correct answer as fine-tuning data. In cases where the model reaches the wrong conclusion, it is provided with the correct answer as a hint and asked to generate a supporting rationale in a process called rationalization. If the model reaches the correct answer, the generated rationale is also added to the fine-tuning data, although it is not independently verified as logically sound. Lastly, the model is fine-tuned and the rationale annotation cycle is repeated.

Least-to-most prompting

Zhou *et al* [46] introduce least-to-most prompting, a few-shot strategy designed to improve easy-to-hard generalization. The method first asks the LLM to decompose a complex problem into an ordered sequence of simpler subproblems. Then the subproblems are solved sequentially, appending earlier answers to the context so that later steps can build on earlier ones. Both decomposition and solution are elicited through prompting alone, without training or fine-tuning. The key distinction from ordinary CoT is that least-to-most prompting explicitly externalizes the decomposition and solves intermediate subproblems one at a time rather than generating a single linear rationale from scratch.

ReAct

Yao *et al* [37] introduce ReAct, a prompting paradigm in which an LLM interleaves free-form reasoning traces with task-specific actions and observations from an external environment. The reasoning traces help the agent decompose goals, track progress, and revise its plan, while the actions allow the agent to gather information or alter the environment. A *thought* is considered a type of action that generates a reasoning trace and updates the current context, but does not affect the external environment. The agent alternates between reasoning and acting, with observations returned by the environment and appended to the context after external actions.

PAL: Program-aided Language Models

Gao *et al* [9] introduce PAL (Program-Aided Language Models), a few-shot prompting method in which the LLM translates a natural-language problem into executable code. The generated program serves as the intermediate reasoning representation, and a Python interpreter executes the program to produce the final answer. This separates semantic decomposition, handled by the LLM, from deterministic computation, handled by the interpreter. The central idea is that LLMs are often better at expressing the solution procedure than reliably executing it.

Reasoning teachers

Ho *et al* [12] propose fine-tune-CoT, a method for transferring chain-of-thought reasoning from a large teacher model to smaller student models. First, a large model is prompted to generate rationales and final answers for labeled training questions. The generated examples are then filtered according to whether the final answer matches the ground-truth. The student model is fine-tuned to generate the teacher-produced rationale followed by the final answer, given the original question as input. Performance improves with more training data, greater student-model capacity, a stronger teacher model, and more diverse teacher-generated rationales.

Multimodal chain-of-thought

Zhang *et al* [43] introduce multimodal-CoT, a two-stage framework for CoT reasoning over language and image inputs. They first show that naively fine-tuning a small language model to generate both a rationale (R) and answer (A), in either order, underperforms directly generating A. They conclude that generated rationales can harm performance

when not grounded in the visual input. To address this, they develop a two-model pipeline. The first model is trained to generate R based on the question, context, answer choices, and image features. The second model is trained to generate A based on the concatenation of the original textual inputs with R plus the original image features. Both models are fine-tuned against gold-standard annotations.

Toolformer

Schick *et al* [22] introduce Toolformer, a self-supervised method for teaching an LM to use external tools through simple API calls. Starting from a small number of demonstrations for each API, the model inserts candidate tool calls into ordinary text, executes those calls, and retains only the calls whose returned results reduce the model’s loss on subsequent tokens. The retained calls are interleaved with the original corpus, and the LM is fine-tuned on this augmented dataset using a standard language-modeling objective. At inference time, generation is paused whenever the model emits an API call, the requested tool is executed, and the returned result is inserted into the context before decoding resumes.

Reflexion

Shinn *et al* [24] introduce Reflexion, an agentic framework composed of an actor, an evaluator, and a self-reflection model. The actor is an LLM-based policy that generates text or actions, while the evaluator assigns a task-specific reward signal to the resulting trajectory. The self-reflection model is an LLM that reviews the trajectory and reward signal, then generates specific, actionable feedback in the form of natural language. This feedback is stored in persistent memory and appended to the actor’s prompt during subsequent trials.

Self-refine

Madaan *et al* [18] introduce Self-Refine, an inference-time framework in which a single LLM acts as generator, critic, and editor without additional training or reinforcement learning. The model first produces an initial response, then uses a separate prompt to generate specific and actionable feedback. Given the input, current response, and feedback, the model is then asked to produce a refined response. This feedback–refinement cycle repeats, with the process history retained in the prompt, until a stopping condition is reached. A limitation is that refinement depends on the model’s ability to correctly identify defects in its own output; when error diagnosis is poor, performance gains are limited.

Tree of thought

Yao *et al* [36] introduce Tree of Thoughts (ToT), an inference-time framework that casts reasoning as a tree search. Each node contains the problem input and the sequence of intermediate thoughts generated so far. At each step, the LLM generates several candidate continuations and evaluates the resulting states, either by assigning each state a direct value judgment or by comparatively voting for the most promising option. An external controller (i.e., a program) keeps track of the tree states and executes the search.

Unlike self-consistent CoT, which aggregates only final answers across independent linear trajectories, ToT evaluates alternatives during the reasoning process itself.

Multi-agent debate

Du *et al* [6] introduce multiagent debate, an inference-time framework in which several stochastic instances of an LLM independently generate candidate responses and reasoning traces. In subsequent rounds, each instance receives the other agents’ current responses as context and generates an updated answer, allowing alternative solution paths to be cross-examined before convergence. Unlike self-consistency, which aggregates final answers by voting, debate repeatedly supplies the full reasoning traces back into the model for iterative synthesis and revision. The main experiments use copies of the same model with the same prompt, so the method is better viewed as iterative multi-sample refinement than as debate among agents with persistent private information or distinct roles.

Verify step-by-step

Lightman *et al* [16] compare outcome-supervised reward models (ORMs), which score complete solutions, with process-supervised reward models (PRMs), which evaluate each intermediate reasoning step. Holding the solution generator fixed, they use each reward model to rank multiple sampled CoT solutions and select the highest-scoring candidate. The PRM is trained using human step-level annotations, with active learning used to prioritize plausible but incorrect solutions for labeling. The PRM produces more reliable rankings than the ORM, in part because it identifies where an error occurs rather than merely whether the final answer is correct. Potential applications of a PRM include weighting sampled solutions in self-consistency, scoring partial states in ToT, and providing the reward signal for RL-based fine-tuning of the generator.

Show, search, and tell

Wu and Xie [32] introduce SEAL (show, search, and tell), a multimodal system that augments a LLaVA-style VLM with an explicit visual-search mechanism. SEAL consists of three interacting components: the VLM, the V^* visual-search module, and visual working memory (VWM). The VWM is structured context containing the question, global image, searched target crops, and target coordinates. The VLM first attempts to answer directly from the global image; if the evidence is insufficient, it identifies missing target objects that must be localized. V^* then performs the visual search and returns the relevant crops and coordinates to the VWM. SEAL differs from later SFT/RL tool-enabled systems because the visual-search behavior is implemented as a structured multi-component pipeline rather than learned policy.

Quiet-STaR

Zelikman *et al* [39] introduce Quiet-STaR, a generalization of STaR that trains a language model to generate internal textual rationales from ordinary pretraining text rather than curated QA examples. After each token position, the model samples several fixed-length thoughts delimited by learned start- and end-of-thought tokens. A parallel attention

scheme allows these counterfactual thoughts to be generated simultaneously across positions. A learned mixing head interpolates between the model’s predictions with and without the thought, while teacher forcing evaluates how well each thought improves prediction of several subsequent ground-truth tokens, rather than only the immediately following token. The model is trained with REINFORCE, assigning each thought a reward based on its improvement in future-token log-likelihood, relative to the other thoughts sampled at the same position, together with the ordinary language-modeling loss. In this way, Quiet-STaR replaces STaR’s final-answer correctness signal with improvement in predicting future text.

Visual sketchpad

Hu *et al* [13] introduce Visual Sketchpad, an agentic system that equips a vision-language model with a workspace and callable tools for producing intermediate visual artifacts. The method follows a ReAct-style loop in which the model generates a thought, writes code that calls tools to manipulate the visual input, observes the resulting sketch, and continues reasoning until ready to emit a final answer. The available tools include ordinary plotting libraries for mathematical reasoning as well as specialist vision modules for object detection, segmentation, depth estimation, cropping, zooming, and image overlay. Visual sketchpad extends text-based CoT and tool-use methods by making the intermediate reasoning state visual rather than purely textual.

Test-time Scaling

Snell *et al* [25] study **test-time scaling**: using additional inference compute to improve the answer to a fixed question without changing the model parameters. They compare three ways of allocating that compute:

- **Parallel sampling** independently generates several complete solutions, then selects an answer by majority vote or with a verifier.
- **Sequential revision** generates a complete solution, then repeatedly produces new complete solutions conditioned on previous attempts.
- **PRM-guided beam search** constructs a solution step by step: several partial reasoning paths are proposed, a process reward model (PRM) scores how likely each prefix is to lead to a correct final answer, and only the highest-scoring paths are continued.

The PRM is trained without human labels by sampling multiple continuations from each partial solution and using the fraction that reach the correct answer as a soft target. Thus, sequential revision operates across complete attempts, whereas beam search chooses among competing partial paths within a solution.

The main result is that the best strategy depends on the question’s difficulty relative to the base model. Difficulty is estimated by generating 2,048 candidate solutions, averaging their PRM scores, and assigning the question to a difficulty bin. Easier questions benefit from longer revision chains because an initial attempt is often nearly correct. For more difficult but still solvable questions, it is better to divide the budget among multiple

independent chains with several revisions per chain: the chains provide different starting approaches, while revision improves each approach. The final answer is selected by scoring attempts within each chain and then comparing the best-supported answers across chains.

Test-time scaling is useful only when the model can already generate viable reasoning paths: sampling can find them, revision can improve them, and PRM-guided search can prioritize them. When such paths are rare, a larger or better-trained model becomes more effective.

LLaVA-CoT

Xu *et al* [33] introduce LLaVA-CoT, a VLM fine-tuned to perform structured multimodal CoT reasoning using a four-stage format: summary, caption, reasoning, and conclusion. GPT-4o is used to construct a dataset, LLaVA-CoT-100k, in which each response is annotated with these tagged stages, and Llama-3.2-11B-Vision-Instruct is then fine-tuned to use this reasoning structure for visual QA. Stage-wise retracing search (SWIRES) is introduced as an inference-time strategy for improving reasoning quality. Candidate outputs are sampled stage-wise, corresponding to the four stages of the structured response. After each stage, a separate multimodal reward model scores the partial responses. Unlike ordinary beam search, which expands partial generations without revisiting earlier semantic units, SWIRES can retrace and regenerate a previous stage when all current-stage candidates are deemed to be low quality.

Coconut

Hao *et al* [11] introduce Chain of Continuous Thought (Coconut), which allows a decoder-only language model to reason through continuous hidden states rather than requiring every intermediate computation to be discretized into language tokens. Let $e_i = E(x_i)$ denote the embedding of the token at position i . Given an input sequence $(x_1, \dots, x_t)$, a standard language model computes the final-layer hidden state at position t via:

$$h_t = f_\theta(e_1, \dots, e_t).$$

Here $f_\theta(\cdot)$ denotes the final-layer hidden state at the last sequence position. The distribution of the next language token is typically given by:

$$p(x_{t+1} \mid x_{\leq t}) = \text{softmax}(Wh_t).$$

Coconut alternates between a language mode and a latent mode, delimited by special `<bot>` and `<eot>` tokens. Within latent mode, projection of h_t to a discrete language token x_{t+1} is bypassed. Instead, the final hidden state h_t is appended directly to the sequence of input embeddings at the next position $z_{t+1} = h_t$. If latent reasoning begins after position t , then after $k - 1$ steps of latent reasoning, the context sequence is:

$$[e_1, \dots, e_t, z_{t+1}, \dots, z_{t+k-1}],$$

and the next latent state is:

$$z_{t+k} = h_{t+k-1} = f_\theta(e_1, \dots, e_t, z_{t+1}, \dots, z_{t+k-1}).$$

The model is trained using a multi-stage curriculum initialized from explicit CoT supervision. In the zeroth stage, the model is trained on examples where `<bot><eot>` are inserted between the prompt and language space CoT. In the first stage, the first text token is ablated and c latent thoughts are inserted. Training uses ordinary next-token cross-entropy on the remaining textual reasoning and answer tokens, while losses on the question and latent positions are masked; consequently, the continuous thoughts are not trained to reconstruct the omitted language. The process of ablating textual tokens and adding c tokens to the latent budget continues until all textual CoT tokens have been removed.

At inference, the model enters latent mode after the question, performs a prescribed number of recurrent latent steps, exits through `<eot>`, and resumes ordinary autoregressive decoding to produce the answer.

Mulberry

Yao *et al* [35] introduce Mulberry, a family of MLLMs trained on reasoning traces generated by Collective Monte Carlo Tree Search (CoMCTS). The goal is to construct multimodal reasoning data with explicit intermediate steps rather than relying on direct-answer prediction or ordinary CoT prompting. CoMCTS differs from standard MCTS in two important ways. First, expansion is performed collectively by multiple MLLMs, each proposing candidate continuations. Second, the paper’s simulation step is not a conventional rollout from a node to a terminal outcome; instead, the model ensemble evaluates candidate reasoning nodes with an evaluation prompt, treats low-scoring nodes as errors, and prunes them together with their descendants. The resulting tree contains both effective reasoning paths and lower-valued sibling nodes; the latter are used to construct synthetic reflection trajectories, where the model inserts a lower-scoring step, reflects, then course corrects. The effective and reflective trajectories form Mulberry-260K, which is then used for supervised fine-tuning rather than RL. Ablations suggest that collective search improves search success over single-model search, and that adding reflection data modestly improves downstream accuracy.

DeepSeek-R1

DeepSeek-AI [3] introduce DeepSeek-R1-Zero, which applies outcome-based RL directly to a pretrained base model without an initial supervised fine-tuning stage, using rule-based rewards for correctness and output formatting. The model is trained with GRPO, an alternative to PPO that compares rewards among groups of sampled responses without requiring a separate value model. Under this objective, longer CoT traces and behaviors such as reflection, verification, and exploration of alternatives emerge without direct supervision of the reasoning process. DeepSeek-R1-Zero shows that CoT reasoning can emerge without direct SFT on CoT examples; however, the subsequent DeepSeek-R1 model included initial SFT.

ReFocus

Fu *et al.* [8] introduce ReFocus, an inference-time framework for structured image understanding in which a multimodal LLM generates Python code to visually edit tables

or charts before question-answering. Given an image and question, the model identifies task-relevant regions or features of the image, then applies simple edits such as masking irrelevant regions or highlighting relevant ones. Unlike Visual Sketchpad, the editing tools do not add external perceptual information; they only modify the existing image to increase its task relevance. Following STaR [38], ReFocus is used to construct a dataset for SFT: if ReFocus generates the correct final answer, the image-question pair, CoT reasoning, edited image, and final answer are retained as a training example. If ReFocus generates an incorrect final answer, the model is prompted once more with the correct answer as a hint. If ReFocus still generates an incorrect final answer, the example is discarded.

Recurrent Depth

Geiping *et al.* [10] propose a depth-recurrent language model that scales test-time computation through latent iteration rather than longer verbalized chains of thought. The architecture consists of a prelude P that maps the input tokens to contextual encodings $e = P(x)$, a shared recurrent block R , and a coda C that maps the final latent state to vocabulary logits. The initial latent state s_0 is sampled from Gaussian noise, and at each recurrence the fixed input encoding e is concatenated with the current state s_{i-1} along the channel dimension, projected back to the model width, and processed by the recurrent block to produce s_i ; repeatedly injecting e anchors the iterative dynamics to the input despite the random initialization.

During pretraining, the recurrence depth r is sampled stochastically for each sequence, encouraging the same recurrent computation to operate effectively over a range of depths. At inference, r may be fixed or selected adaptively by stopping when the KL divergence between the next-token distributions at successive depths falls below a threshold. The architecture also naturally supports **speculative decoding**, in which a cheaper model proposes a block of future tokens and a more accurate model verifies those proposals in parallel: here, the same recurrent network can act as both models, with shallow recurrence serving as the inexpensive proposer and deeper recurrence serving as the verifier.

Visual-RFT

Liu *et al* [17] introduce Visual-RFT, which applies GRPO-style reinforcement fine-tuning to visual perception tasks using automatically computable rewards. The method samples multiple responses containing a `<think>` block and a final `<answer>` block, then updates the model based on task-specific rewards. For classification, the reward is based on whether the predicted class matches the ground-truth; for detection and grounding, the reward combines IoU-based localization, confidence calibration, and format compliance. If the model assigns confidence $C \in [0, 1]$ to a predicted box, the confidence reward is C when the model is correct and $1 - C$ when the model is incorrect. The paper’s methodological contribution is the adaptation of verifiable-reward RL to visual tasks with limited fine-tuning data.

Vision-R1

Huang *et al* [14] introduce Vision-R1, a multimodal reasoning model motivated by the observation that directly applying GRPO to a base MLLM is not sufficient to induce complex multimodal reasoning. To cold-start reasoning, they construct a multimodal CoT dataset through modality bridging. First, an MLLM generates a caption and initial reasoning from an image-question-answer example. Next, the original image, question, and generated reasoning are then passed back to an MLLM to produce a detailed text-only description. Finally, DeepSeek-R1 uses the question and description to generate the CoT target. The resulting examples are filtered by final-answer correctness and rule-based checks, then used for SFT, although natural alternatives to this pipeline are not systematically explored. After SFT, the authors observe an overthinking problem in which the model produces long but often unnecessary or incorrect reasoning chains. They address this with Progressive Thinking Suppression Training, a staged GRPO procedure that first imposes a shorter output-token budget and then progressively relaxes it.

MM-Eureka

Meng *et al* [19] propose MM-Eureka, a multimodal reasoning model trained directly by RL. They introduce the open-source MMK12 dataset, which contains real-world K12 multimodal math problems with verified answers and solution processes. The training task is fill-in-the-blank, and training is performed using GRPO with simple accuracy and format rewards. Online filtering is employed to remove examples where all or none of the sampled responses are correct, since such examples provide no advantage signal. For the larger 32B-parameter model, training is divided into two stages. In the first, no KL regularization is applied, encouraging exploration; in the second stage, KL regularization is restored to prevent model collapse. An important conceptual conclusion is that RL does not primarily teach new knowledge, but instead increases the model’s ability to correctly deploy existing knowledge.

R1-OneVision

Yang *et al* [34] propose R1-OneVision, a multimodal reasoning model built around cross-modal formalization. The main idea is to convert the task-relevant content of an image into structured textual representations, such as captions, bounding boxes, tables, diagrams, or code-like descriptions, then use language-model reasoning over this representation. These formalized image descriptions are used to generate multimodal chain-of-thought training data, followed by supervised fine-tuning and rule-based GRPO with accuracy and format rewards.

VisualPRM

Wang *et al* [28] introduce VisualPRM, a multimodal process reward model (PRM) for evaluating visual reasoning traces. Unlike methods that directly update the policy MLLM, VisualPRM trains a separate critic. Given an image, a question, and a reasoning prefix, the critic predicts whether the current step leaves the partial solution on a path that can reach the correct final answer. Training supervision is generated automatically by Monte Carlo continuation. For a solution $s = (s_0, \dots, s_n)$, each prefix $s_{\leq i}$ is

completed multiple times, and the expected accuracy of step i is estimated as:

$$r_i = \frac{\#\{\text{correct completions from } s_{\leq i}\}}{\#\{\text{sampled completions from } s_{\leq i}\}}.$$

The value-based PRM labels a step as correct if $r_i > 0$, while the advantage-based PRM labels a step according to whether $r_i - r_{i-1}$ is positive, zero, or negative. Empirically, the value-based formulation performs better, likely because the advantage target is noisier under automatically generated labels. Human step-level annotations are not required for training the PRM; they are used separately to construct VisualProcessBench, a benchmark for evaluating whether multimodal critics can detect incorrect reasoning steps. At inference time, VisualPRM is used for Best-of- N selection: the policy model samples multiple candidate reasoning traces, the critic scores them step by step, and the highest-scoring solution is selected.

R1-VL

Zhang *et al* [40] introduce R1-VL, a multimodal reasoning model trained with Step-wise Group Relative Policy Optimization (StepGRPO). The method begins with SFT on multimodal CoT traces, then applies GRPO using auxiliary rule-based rewards intended to reduce the sparsity of final-answer rewards. The main reward, Step-wise Reasoning Accuracy Reward (StepRAR), first uses GPT-4 to extract key intermediate steps from the reference CoT, such as essential equations, variables, or facts needed for the solution. For a sampled reasoning trajectory, let f_i denote the fraction of extracted key steps matched by the generated response. The StepRAR reward is:

$$r_i = \begin{cases} 1 + \alpha f_i, & \text{final answer correct,} \\ \alpha f_i, & \text{final answer present but incorrect,} \\ 0, & \text{no final answer.} \end{cases}$$

Thus, a trajectory can receive partial credit for containing useful intermediate reasoning even if it does not reach the correct final answer. The second reward, Step-wise Reasoning Validity Reward (StepRVR), is better understood as a structured-format reward: it checks whether the response contains the expected components, such as image description, rationale, step-by-step reasoning, and final answer, in the expected order. Despite the name, StepRVR is trajectory-level rather than truly dense or step-wise, and it does not verify the substantive correctness of individual reasoning steps. The combined reward is used to compute group-relative advantages for GRPO, providing a lightweight alternative to training a separate process reward model.

VLM-R1

Shen *et al* [23] introduce VLM-R1, a framework for applying R1-style GRPO to VLMs using continuous rewards tied directly to object-localization annotations. The tasks considered require the model to draw bounding boxes around either a single described object or all objects matching a list of queried labels; rewards are based on the overlap between predicted and ground-truth boxes, using IoU for single-object localization and mAP for multi-object detection. In the detection setting, mAP is calculated by matching predicted boxes to ground-truth boxes using class labels and IoU thresholds, constructing a

precision–recall curve from the ranked predictions, computing AP as the area under that curve, and averaging AP across classes.

The naive mAP reward can under-penalize redundant boxes, because emitting several similar boxes increases the chance that at least one nearly matches the ground-truth. It can also encourage spurious detections when classes not present in the image are omitted from the per-image AP calculation. To address this, the reward is modified with a penalty that diminishes the reward when the number of predicted boxes exceeds the number of ground-truth boxes:

$$\min \left(1, \frac{N_{\text{gt}}}{N_{\text{pred}}} \right).$$

Here N_{gt} and N_{pred} are the numbers of ground-truth and predicted boxes, respectively.

OpenThinkIMG

Su *et al* introduce OpenThinkIMG, an end-to-end framework for tool-augmented visual reasoning. A standardized set of vision tools is defined. For each training example (Q_i, I_i) , consisting of a question and an image, GPT-4o generates an action plan:

$$\mathcal{A}_i = (a_1, \dots, a_{n_i}).$$

The actions are executed to produce action-observation trajectories $\tau_i = \{(a_t^{(i)}, o_t^{(i)})\}_{t=1}^{n_i}$. The trajectories are filtered, removing instances that are malformed, incomplete, or logically inconsistent. SFT is performed, optimizing:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \ln \mathbb{P}_{\theta}(\tau_i \mid Q_i, I_i).$$

This initializes the ability to imitate valid tool-use trajectories. Next, groups of tool-use trajectories are sampled from the current policy, the planned actions are executed with external vision tools, and the resulting trajectories are scored by final-answer correctness. V-ToolRL, the proposed algorithm, treats tool invocation and tool observations as part of the policy rollout. The model is updated via GRPO. Empirically, SFT improves over the base model, V-ToolRL improves substantially over SFT and tool-free RL, and the learned policy reduces unnecessary tool calls during training.

MM-PRM

Du *et al* [5] propose MM-PRM, a multimodal process reward model for mathematical reasoning. The main methodological contribution is a scalable pipeline for constructing step-level supervision without human annotation. First, the authors train a structured policy model, MM-Policy, to generate multimodal chain-of-thought solutions with explicitly delimited reasoning steps. They then use a curated set of 10,000 multimodal K-12 math problems with verifiable answers as seed data and apply a Monte Carlo tree search (MCTS)-based continuation procedure to generate step-level targets. Specifically, each intermediate reasoning prefix is assigned a Monte Carlo continuation score, reflecting the probability that rollouts from that prefix lead to the correct final answer. MM-PRM is trained to predict these continuous step-level scores, yielding a verifier that evaluates reasoning traces conditioned on the original multimodal input.

DeepEyes

Zheng *et al* [45] introduce DeepEyes, a vision-language model fine-tuned to perform interleaved multimodal CoT reasoning through RL. The base model is Qwen2.5-VL-7B, and the core visual operation is zoom-in with cropping. Given a question and an initial image, the model can continue reasoning in text, call the zoom-in tool, or emit a final answer. This results in an interleaved sequence of text and image observations:

$$s_t = \{(X_0, I_0), (X_1, I_1), \dots, (X_t, I_t)\},$$

which is treated as the model state. In contrast to OpenThinkIMG [26], DeepEyes avoids SFT, but requires a carefully curated training set. The VLM is fine-tuned via GRPO, with completed rollouts scored according to (i) final-answer correctness, (ii) output-format validity, and (iii) a conditional tool-use reward awarded only when active perception is used in a correct trajectory. As in OpenThinkIMG, tool use becomes more selective over the course of training.

Pixel reasoner

Wang *et al* [27] introduce Pixel Reasoner, a VLM fine-tuned to perform pixel-space reasoning by interleaving textual reasoning with visual operations. The base VLM is given access to two pixel-space operations: zoom-in with crop for images and frame selection for videos. GPT-4o is used to construct reasoning traces following a template in which a visual operation is invoked to localize the relevant region, followed by an analysis of the local visual evidence. Trajectories containing incorrect or suboptimal visual operations are constructed in order to teach self-correction, as are trajectories that omit visual operations entirely. Similar to OpenThinkIMG [26], training consists of SFT followed by GRPO. A reward model is specified that includes (i) final-answer correctness, (ii) a curiosity bonus to encourage exploration of pixel-space reasoning, and (iii) an efficiency penalty to discourage excessive visual operations. Pixel-space reasoning is most beneficial on visually intensive tasks; correctness rewards alone are insufficient for learning effective visual operations; and self-correction trajectories are important for teaching error correction.

Chain-of-focus

Zhang *et al.* [42] introduce Chain-of-Focus, a VLM framework for adaptive visual search. At each step, the model can either answer from the available evidence or emit a bounding box to zoom in on and acquire additional visual information. The method is trained in two stages: SFT on visual-search trajectories, generated by an agentic model, followed by GRPO using an adaptive group-aware reward. Compared with Pixel Reasoner, Chain-of-Focus places more emphasis on adaptive efficiency: its group-aware reward penalizes zooming when another rollout can answer correctly without zoom, encouraging the model to reserve active perception for cases where the original image evidence is insufficient.

GRIT

Fan *et al* [7] propose GRIT, which trains an MLLM to generate a single autoregressive reasoning trace interleaving natural language with bounding-box coordinates. The

method is applied directly to (image, question, answer) triples, without requiring supervised reasoning chains or bounding-box annotations. Training uses GRPO with grounded reasoning (GRPO-GR), which scores grouped candidate traces using final-answer correctness, as judged by GPT-4o, format rewards for valid output formatting, syntactic rewards for the generated bounding boxes, and an optional count-matching reward for counting tasks. A key feature is that emitted boxes are not cropped and re-encoded as new visual inputs; they simply serve as reference points.

VTool-R1

Wu *et al* [31] introduce VTool-R1, a reinforcement fine-tuning framework that trains VLMs to perform multimodal CoT reasoning by interleaving text with intermediate visual edits. Given an image and question, the model either answers directly or emits a Python-based tool call that highlights, masks, or draws boxes around task-relevant rows, columns, or chart elements; the edited image is then fed back alongside the original image for final reasoning. Training uses GRPO with outcome-based rewards on final-answer correctness, optimizing the final response rather than directly supervising or rewarding the intermediate tool call. Empirically, tool use becomes more selective over training, suggesting that the model learns when visual editing is useful rather than simply learning to invoke tools more often.

Active-o3

Zhu *et al* [47] propose ACTIVE-o3, which trains a multimodal LLM as a sensing policy. The policy simultaneously proposes up to three crops in a single forward pass, represented as bounding boxes. A frozen task model then performs object detection or segmentation. Training proceeds via GRPO without initial SFT. The reward combines task-model performance with a heuristic proposal-quality reward for valid formatting, non-overlap, reasonable box size, and coverage.

ViGoRL

Sarch *et al* [21] introduce ViGoRL, a framework for training VLMs to perform visually grounded reasoning. They begin from the standard autoregressive view of a VLM policy, in which a reasoning trajectory is generated step-by-step conditional on the image, query, and previous reasoning steps:

$$\pi_{\theta}(\tau \mid I, q) = \left\{ \prod_{t=1}^T \pi_{\theta}(s_t \mid I, q, s_{<t}) \right\} \pi_{\theta}(a \mid I, q, s_{\leq T})$$

ViGoRL modifies the trajectory representation by replacing each textual reasoning step s_t with a grounded reasoning step (s_t, x_t, y_t) , which ties the reasoning to a specific location in the image.

Training begins with data generation via Monte Carlo tree search (MCTS), in which a strong teacher model searches over grounded reasoning trajectories. Successful paths are linearized for SFT. The MCTS uses UCB at intermediate nodes to decide which partial grounded reasoning chain to extend. The model is then fine-tuned with GRPO, using

final-answer correctness together with format and coordinate-validity rewards to reinforce grounded reasoning traces.

A multi-turn variant is also considered in which the model can call a crop tool at designated coordinates, receive zoomed-in visual observations, and continue reasoning with this additional evidence. Without explicit visual grounding, VLMs tend to produce abstract reasoning, whereas with visual grounding, training shifts reasoning toward concrete inspection of the image.

RRVF

Chen *et al* [2] propose Reasoning–Rendering–Visual-Feedback (RRVF), a framework for training multimodal models on image-to-code tasks without paired ground-truth code. Given a chart or webpage image, the model iteratively generates to reproduce the image, executes it, then revises the code using natural-language descriptions of the discrepancies between the rendered and target images. A separate visual judge scores the final rendering, and GRPO optimizes a trajectory-level reward combining visual similarity, format correctness, and successful tool use. For chart reconstruction, a small SFT cold start first teaches the multi-turn tool-calling protocol, after which tool use and task solving are optimized jointly during RL. RRVF improves code executability and visual fidelity over the base model and an image–code SFT baseline.

Vision-SR1

Li *et al* propose Vision-SR1, a self-rewarded RL method that decomposes VLM output into a visual description, a language reasoning trace, and a final answer. Each image–question pair is examined twice. In the first rollout, the model receives the image and question, then generates the description, CoT reasoning, and final answer. Correctness of the final answer provides the answer reward. In the second rollout, the same model is re-prompted with only the question and the first-stage visual description. Correctness of the final answer provides a self-reward for producing a visual description sufficient for answering. Training uses GRPO-style optimization, with separate advantage scores calculated for the first and second rollouts. The method does not acquire new visual evidence through crops or tools; rather, it tries to make the model’s internal visual perception more self-contained and less dependent on language shortcuts.

Perception before reasoning

Chen *et al.* [1] introduce PeBR-R1, a two-stage RL framework for improving visual reasoning in VLMs. The central premise is that VLMs should first be optimized to describe the relevant visual content before being optimized to reason from that content. A model that misperceives the image may still produce plausible but incorrect reasoning. Data are drawn primarily from Mulberry-260K, augmented with targeted additional examples. Prior to GRPO-based RL, 8 rollouts from the baseline model are generated for each question, and questions are categorized as easy, medium, or hard depending on whether all, some, or none of the final answers are correct. An initial SFT stage teaches the model to produce responses in the target format, including an image description, rationale, step-by-step reasoning, and final answer. RL is then divided into two stages: perception RL

and reasoning RL. Perception RL uses easy-difficulty questions and rewards the image description using image-text alignment, teacher-derived keyword overlap, format validity, and a length penalty. Reasoning RL uses medium-difficulty questions and rewards final-answer correctness and output format. Ablation studies show that this staged training regimen outperforms standard GRPO and a single-stage approach that intermixes perception and reasoning rewards.

Latent Visual Reasoning

Most VLMs either *think about images*, by performing CoT reasoning in text space while conditioning on a fixed visual input, or *think with images*, by iteratively editing and re-encoding the image. Li *et al* [15] propose Latent Visual Reasoning (LVR), a multimodal reasoning framework in which an MLLM alternates between ordinary text generation and latent visual-reconstruction segments. Within the latent segments, the final hidden state is fed directly back into the model as the next input embedding, as in Coconut [11]. During SFT, a query-relevant region of interest is identified by a bounding box, and the latent hidden states are trained to reconstruct the corresponding projected visual tokens, while the subsequent textual answer is trained with the usual next-token prediction loss. The second training stage applies a modified GRPO objective in which rewards depend on answer accuracy and response format. Since learned stopping mechanisms for variable-length latent reasoning are unstable, the main experiments use a fixed number of latent steps. Although LVR improves performance on several perception-intensive benchmarks, it is unclear if the improvements are due to latent-space reasoning or simply resurfacing query-relevant visual tokens.

Absolute Zero

Zhao *et al.* [44] introduce Absolute Zero, a paradigm in which a pretrained language model generates its own reasoning tasks and learns to solve them without externally curated examples. Their implementation, the Absolute Zero Reasoner (AZR), uses a Python interpreter to validate proposed tasks, compute their correct answers, and verify the solver’s responses. A single model acts as both proposer and solver. The proposer first generates a task

$$\tau \sim \pi_{\theta}^{\text{propose}}(\cdot \mid z),$$

where z may contain previously generated examples. The environment then validates the proposal and constructs a solver query and verified answer,

$$(x, y^*) \sim f_e(\cdot \mid \tau).$$

The solver produces a response

$$y \sim \pi_{\theta}^{\text{solve}}(\cdot \mid x),$$

after which the proposer receives a learnability reward $r_{\text{propose}}(\tau, \pi_{\theta})$ and the solver receives a correctness reward $r_{\text{solve}}(y, y^*)$. Both roles are optimized jointly.

AZR generates tasks from a program–input–output triplet (p, i, o) satisfying $o = p(i)$:

- **Deduction:** predict the output o from the program p and input i .
- **Abduction:** find an input i such that $p(i) = o$ for a specified output o .
- **Induction:** synthesize a program p from input–output examples and a natural-language description, with held-out examples used for verification.

Using the empirical solver success rate $\bar{r}_{\text{solve}}$, the paper defines

$$r_{\text{propose}} = \begin{cases} 0, & \bar{r}_{\text{solve}} = 0, \\ 1 - \bar{r}_{\text{solve}}, & \bar{r}_{\text{solve}} > 0. \end{cases}$$

Although described as favoring moderately difficult tasks, this reward instead favors the hardest tasks with nonzero empirical success. A more direct competence-boundary reward would be

$$r_{\text{propose}} = \bar{r}_{\text{solve}} (1 - \bar{r}_{\text{solve}}),$$

which is zero for uniformly solved or unsolved tasks and maximized when the solver succeeds half the time.

AZR is trained with task-relative REINFORCE++ rather than GRPO. GRPO estimates a prompt-specific baseline from multiple responses to the same question, whereas AZR uses one training rollout per prompt and normalizes rewards separately across the six combinations of three task types and two roles:

$$A_{\text{task,role}}^{\text{norm}} = \frac{r - \mu_{\text{task,role}}}{\sigma_{\text{task,role}}}.$$

This reduces sampling cost and accommodates heterogeneous reward distributions, but controls less directly for individual-question difficulty. AZR may nevertheless be well suited to GRPO because it already samples multiple solver responses to estimate task difficulty: the same rollouts could provide within-prompt solver advantages, while their reward variance, proportional to $\bar{r}_{\text{solve}}(1 - \bar{r}_{\text{solve}})$ for binary outcomes, could train the proposer to generate informative prompts.

Chain of Visual Thought

Qin *et al* [20] propose Chain-of-Visual-Thought (CoVT), a framework that augments VLMs with autoregressively generated continuous visual tokens supervised by specialized vision models. The method introduces separate token families for segmentation, depth, edges, and DINO features, using SAM, DepthAnything v2, PIDINet, and DINOv2 as perceptual teachers. Segmentation, depth, and edge tokens are projected into the corresponding expert’s prompt or readout space and trained through reconstruction of dense masks, depth maps, or edge maps, while DINO tokens are aligned directly to patch-level features. The VLM is optimized with the usual next-token cross-entropy loss together with these auxiliary visual losses, using a four-stage SFT curriculum that first teaches the model to interpret the visual tokens, then generate them, incorporate them into a `{think}` block before answering, and finally operate with randomly omitted subsets of token types. At inference, the VLM generates the visual tokens itself and conditions

the final answer on them, while dense decoding using the corresponding expert encoders and decoders is optional and used primarily for visualization. CoVT improves several perception-intensive benchmarks, although the method does not establish that the generated tokens alone contain the dense perceptual information, since reconstruction also depends on expert encoder features.

ToolsRL

Dong *et al* [4] propose ToolsRL, a two-stage reinforcement-learning framework that decouples learning how to use visual tools from learning when those tools improve task performance. Rather than collecting full expert tool-use trajectories, they derive inexpensive supervision for individual operations: bounding boxes for zooming, known transformations for rotation and flipping, and target coordinates for drawing points or lines. In Stage 1, GRPO combines a global tool reward, which credits the best tool call anywhere in the trajectory, with an answer-conditioned tool reward, which scores whether the model can identify which of its own tool calls correctly manipulated the image. In Stage 2, tool rewards are removed and GRPO optimizes final-answer accuracy equipped with the full learned toolbox. This separation prevents answer optimization from driving the model toward the easier shortcut of text-only reasoning before tool use has been acquired. Overall, the framework converts narrowly instrumented toy tasks into reusable action primitives that can be deployed on more difficult spatial-reasoning problems. Empirically, the staged curriculum produces more frequent, compositional, and effective tool use than answer-only RL, generic tool-use bonuses, or jointly optimized tool and answer rewards.

Perceval

Min *et al* propose Perceval, a perception-centric process reward model that identifies image-related hallucinations within a VLM’s reasoning trace. Given an image, question, and response, Perceval checks visual claims against the image and identifies spans that contain perceptual errors. Perceval is trained by SFT on examples requiring fine-grained visual grounding, using Qwen2.5-VL backbones at 3B and 7B scales. During GRPO, tokens within the identified spans are selectively penalized, replacing the uniform sequence-level advantage with token-specific advantages. During inference, Perceval can truncate a response before the first erroneous span and ask the policy model to regenerate from the prefix. The key idea is training a visually-grounded PRM to enable localized credit assignment and targeted correction of perceptual errors, rather than assigning a single advantage to the entire reasoning trajectory.

Visual grounding

Zhang *et al* [41] propose visually grounded thinking, in which natural-language reasoning is interleaved with `<obj>` tags that name relevant visual objects and attach point or box coordinates. Like ViGoRL, the model is first initialized by SFT on grounded reasoning traces and then optimized with GRPO using answer-correctness and grounding rewards. The training data are synthesized by generating correct reasoning traces, extracting the visual objects they require, and grounding those objects with a SAM3-based agent. During RL, a lightweight VLM matches each generated object mention and its nearby textual

context to a saved target before localization is scored by box IoU or point-level F1. Unmatched generated objects are not penalized, allowing the model to cite additional visual evidence beyond the annotated targets.

References

- [1] Yan Chen, Long Li, Teng Xi, Long Zeng, and Jingdong Wang. Perception before reasoning: Two-stage reinforcement learning for visual reasoning in vision-language models. *arXiv*, 09 2025. <https://arxiv.org/abs/2509.13031>.
- [2] Yang Chen, Yufan Shen, Wenxuan Huang, Sheng Zhou, Qunshu Lin, Xinyu Cai, Zhi Yu, Jiajun Bu, Botian Shi, and Yu Qiao. Learning only with images: Visual reinforcement learning with reasoning, rendering, and visual feedback. *arXiv*, 07 2025. <https://arxiv.org/abs/2507.20766>.
- [3] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv*, 01 2025. <https://arxiv.org/abs/2501.12948>.
- [4] Qihua Dong, Gozde Sahin, Pei Wang, Zhaowei Cai, Robik Shrestha, Hao Yang, and Davide Modolo. Visual reasoning through tool-supervised reinforcement learning. *arXiv*, 04 2026. <https://arxiv.org/abs/2604.19945>.
- [5] L Du, F Meng, Z Liu, Z Zhou, P Luo, Q Zhang, and W Shao. Mm-prm: Enhancing multimodal mathematical reasoning with scalable step-level supervision. *arXiv*, 05 2025. <https://arxiv.org/abs/2505.13427>.
- [6] Y Du, S Li, A Torralba, J B Tenenbaum, and I Mordatch. Improving factuality and reasoning in language models through multiagent debate. *arXiv*, 05 2023. <https://arxiv.org/abs/2305.14325>.
- [7] Yue Fan, Xuehai He, Diji Yang, Kaizhi Zheng, Ching-Chen Kuo, Yuting Zheng, Sravana Jyothi Narayanaraju, Xinze Guan, and Xin Eric Wang. GRIT: Teaching MLLMs to think with images. *arXiv*, 05 2025. <https://arxiv.org/abs/2505.15879>.
- [8] X Fu, M Liu, Z Yang, J Corring, Y Lu, J Yang, D Roth, D Florencio, and C Zhang. Refocus: Visual editing as a chain of thought for structured image understanding. *arXiv*, 01 2025. <https://arxiv.org/abs/2501.05452>.
- [9] L Gao, A Madaan, S Zhou, U Alon, P Liu, Y Yang, J Callan, and G Neubig. PAL: Program-aided language models. *arXiv*, 11 2022. <https://arxiv.org/abs/2211.10435>.
- [10] Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. *arXiv*, 02 2025. <https://arxiv.org/abs/2502.05171>.
- [11] Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space. *arXiv*, 12 2024. [arXiv:2412.06769](https://arxiv.org/abs/2412.06769).
- [12] N Ho, L Schmid, and S-Y Yun. Large language models are reasoning teachers. *arXiv*, 12 2022. <https://arxiv.org/abs/2212.10071>.

- [13] Y Hu, W Shi, X Fu, D Roth, M Ostendorf, L Zettlemoyer, N A Smith, and R Krishna. Visual sketchpad: Sketching as a visual chain of thought for multimodal language models. *arXiv*, 06 2024. <https://arxiv.org/abs/2406.09403>.
- [14] Wenxuan Huang, Bohan Jia, Zijie Zhai, Shaosheng Cao, Zheyu Ye, Fei Zhao, Zhe Xu, Yao Hu, and Shaohui Lin. Vision-R1: Incentivizing reasoning capability in multimodal large language models. *arXiv*, 03 2025. <https://arxiv.org/abs/2503.06749>.
- [15] Bangzheng Li, Ximeng Sun, Jiang Liu, Ze Wang, Jialian Wu, Xiaodong Yu, Hao Chen, Emad Barsoum, Muhao Chen, and Zicheng Liu. Latent visual reasoning. *arXiv*, 09 2025. <https://arxiv.org/abs/arXiv:2509.24251>.
- [16] H Lightman, V Kosaraju, Y Burda, H Edwards, B Baker, T Lee, J Leike, J Schulman, I Sutskever, and K Cobbe. Let’s verify step by step. *arXiv*, 05 2023. <https://arxiv.org/abs/2305.20050>.
- [17] Ziyu Liu, Zeyi Sun, Yuhang Zang, Xiaoyi Dong, Yuhang Cao, Haodong Duan, Dahua Lin, and Jiaqi Wang. Visual-RFT: Visual reinforcement fine-tuning. *arXiv*, 03 2025. <https://arxiv.org/abs/2503.01785>.
- [18] A Madaan, N Tandon, P Gupta, S Hallinan, L Gao, S Wiegrefe, U Alon, N Dziri, S Prabhumoye, Y Yang, S Gupta, B P Majumder, K Hermann, S Welleck, A Yazdanbakhsh, and P Clark. Self-refine: Iterative refinement with self-feedback. *arXiv*, 03 2023. <https://arxiv.org/abs/2303.17651>.
- [19] F Meng, L Du, Z Liu, Z Zhou, Q Lu, D Fu, T Han, B Shi, W Wang, J He, K Zhang, P Luo, Y Qiao, Q Zhang, and W Shao. Mm-eureka: Exploring the frontiers of multimodal reasoning with rule-based reinforcement learning. *arXiv*, 03 2025. <https://arxiv.org/abs/2503.07365>.
- [20] Yiming Qin, Bomin Wei, Jiaxin Ge, Konstantinos Kallidromitis, Stephanie Fu, Trevor Darrell, and XuDong Wang. Chain-of-visual-thought: Teaching VLMs to see and think better with continuous visual tokens. *arXiv*, 11 2025. <https://arxiv.org/abs/2511.19418>.
- [21] Gabriel Herbert Sarch, Snigdha Saha, Naitik Khandelwal, Ayush Jain, Michael J. Tarr, Aviral Kumar, and Katerina Fragkiadaki. Grounded reinforcement learning for visual reasoning. *arXiv*, 05 2025. <https://arxiv.org/abs/2505.23678>.
- [22] T Schick, J Dwivedi-Yu, R Dessi, R Raileanu, M Lomeli, L Zettlemoyer, N Cancedda, and T Scialom. Toolformer: Language models can teach themselves to use tools. *arXiv*, 02 2023. <https://arxiv.org/abs/2302.04761>.
- [23] Haozhan Shen, Peng Liu, Jingcheng Li, Chunxin Fang, Yibo Ma, Jiajia Liao, Qiaoli Shen, Zilun Zhang, Kangjia Zhao, Qianqian Zhang, Ruochen Xu, and Tiancheng Zhao. VLM-R1: A stable and generalizable R1-style large vision-language model. *arXiv*, 04 2025. <https://arxiv.org/abs/2504.07615>.
- [24] N Shinn, F Cassano, E Berman, A Gopinath, K Narasimhan, and S Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv*, 03 2023. <https://arxiv.org/abs/2303.11366>.

- [25] C Snell, J Lee, K Xu, and A Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv*, 08 2024. <https://arxiv.org/abs/2408.03314>.
- [26] Z Su, L Li, M Song, Y Hao, Z Yang, J Zhang, G Chen, J Gu, J Li, X Qu, and Y Cheng. Openthinking: Learning to think with images via visual tool reinforcement learning. *arXiv*, 05 2025. <https://arxiv.org/abs/2505.08617>.
- [27] H Wang, A Su, R Weimin, F Lin, and W Chen. Pixel reasoner: Incentivizing pixel-space reasoning with curiosity-driven reinforcement learning. *arXiv*, 05 2025. <https://arxiv.org/abs/2505.15966>.
- [28] Weiyun Wang, Zhangwei Gao, Lianjie Chen, Zhe Chen, Jinguo Zhu, Xiangyu Zhao, Yangzhou Liu, Yue Cao, Shenglong Ye, Xizhou Zhu, Lewei Lu, Haodong Duan, Yu Qiao, Jifeng Dai, and Wenhai Wang. VisualPRM: An effective process reward model for multimodal reasoning. *arXiv*, 03 2025. <https://arxiv.org/abs/2503.10291>.
- [29] X Wang, J Wei, D Schuurmans, Q Le, E Chi, S Narang, A Chowdhery, and D Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv*, 03 2022. <https://arxiv.org/abs/2203.11171>.
- [30] J Wei, X Wang, D Schuurmans, M Bosma, B Ichter, F Xia, E Chi, Q Le, and D Zhou. Chain-of-thought prompting elicits reasoning in large language models. *arXiv*, 01 2022. <https://arxiv.org/abs/2201.11903>.
- [31] M Wu, J Yang, J Jiang, M Li, K Yan, H Yu, M Zhang, C Zhai, and K Nahrstedt. Vtool-r1: Vlms learn to think with images via reinforcement learning on multimodal tool use. *arXiv*, 05 2025. <https://arxiv.org/abs/2505.19255>.
- [32] Penghao Wu and Saining Xie. V*: Guided visual search as a core mechanism in multimodal llms. *arXiv*, 12 2023. <https://arxiv.org/abs/2312.14135>.
- [33] Guowei Xu, Peng Jin, Hao Li, Yibing Song, Lichao Sun, and Li Yuan. Llava-cot: Let vision language models reason step-by-step. *arXiv*, 11 2024. <https://arxiv.org/abs/2411.10440>.
- [34] Y Yang, X He, H Pan, X Jiang, Y Deng, X Yang, H Lu, D Yin, F Rao, M Zhu, B Zhang, and W Chen. R1-onevision: Advancing generalized multimodal reasoning through cross-modal formalization. *arXiv*, 03 2025. <https://arxiv.org/abs/2503.10615>.
- [35] Huanjin Yao, Jiaying Huang, Wenhao Wu, Jingyi Zhang, Yibo Wang, Shunyu Liu, Yingjie Wang, Yuxin Song, Haocheng Feng, Li Shen, and Dacheng Tao. Mulberry: Empowering MLLM with o1-like reasoning and reflection via collective monte carlo tree search. *arXiv*, 12 2024. <https://arxiv.org/abs/2412.18319>.
- [36] S Yao, D Yu, J Zhao, I Shafran, T L Griffiths, Y Cao, and K Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *arXiv*, 05 2023. <https://arxiv.org/abs/2305.10601>.

- [37] S Yao, J Zhao, D Yu, N Du, I Shafran, K Narasimhan, and Y Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv*, 10 2022. <https://arxiv.org/abs/2210.03629>.
- [38] E Zelikman, Y Wu, J Mu, and N D Goodman. STaR: Bootstrapping reasoning with reasoning. *arXiv*, 03 2022. <https://arxiv.org/abs/2203.14465>.
- [39] Eric Zelikman, Georges Harik, Yijia Shao, Varuna Jayasiri, Nick Haber, and Noah D. Goodman. Quiet-star: Language models can teach themselves to think before speaking. *arXiv*, 03 2024. arXiv:2403.09629.
- [40] Jingyi Zhang, Jiaxing Huang, Huanjin Yao, Shunyu Liu, Xikun Zhang, Shijian Lu, and Dacheng Tao. R1-VL: Learning to reason with multimodal large language models via step-wise group relative policy optimization. *arXiv*, 03 2025. <https://arxiv.org/abs/2503.12937>.
- [41] Junkai Zhang, Yihe Deng, Kai-Wei Chang, and Wei Wang. Thinking with visual grounding. *arXiv*, 06 2026. <https://arxiv.org/abs/2606.16122>.
- [42] Xintong Zhang, Zhi Gao, Bofei Zhang, Pengxiang Li, Xiaowen Zhang, Yang Liu, Tao Yuan, Yuwei Wu, Yunde Jia, Song-Chun Zhu, and Qing Li. Chain-of-focus: Adaptive visual search and zooming for multimodal reasoning via rl. *arXiv*, 05 2025. <https://arxiv.org/abs/2505.15436>.
- [43] Zhuosheng Zhang, Aston Zhang, Mu Li, Hai Zhao, George Karypis, and Alex Smola. Multimodal chain-of-thought reasoning in language models. *arXiv*, 02 2023. <https://arxiv.org/abs/2302.00923>.
- [44] Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Matthieu Lin, Shenzhi Wang, Qingyun Wu, Zilong Zheng, and Gao Huang. Absolute zero: Reinforced self-play reasoning with zero data. *arXiv*, 10 2025. <https://arxiv.org/abs/2505.03335>.
- [45] Z Zheng, M Yang, J Hong, C Zhao, G Xu, L Yang, C Shen, and X Yu. Deepeyes: Incentivizing “thinking with images” via reinforcement learning. *arXiv*, 05 2025. <https://arxiv.org/abs/2505.14362>.
- [46] D Zhou, N Scharli, L Hou, J Wei, N Scales, X Wang, D Schuurmans, C Cui, O Bousquet, Q Le, and E Chi. Least-to-most prompting enables complex reasoning in large language models. *arXiv*, 05 2022. <https://arxiv.org/abs/2205.10625>.
- [47] Muzhi Zhu, Hao Zhong, Canyu Zhao, Zongze Du, Zheng Huang, Mingyu Liu, Hao Chen, Cheng Zou, Jingdong Chen, Ming Yang, and Chunhua Shen. Active-O3: Empowering multimodal large language models with active perception via GRPO. *arXiv*, 05 2025. <https://arxiv.org/abs/2505.21457>.